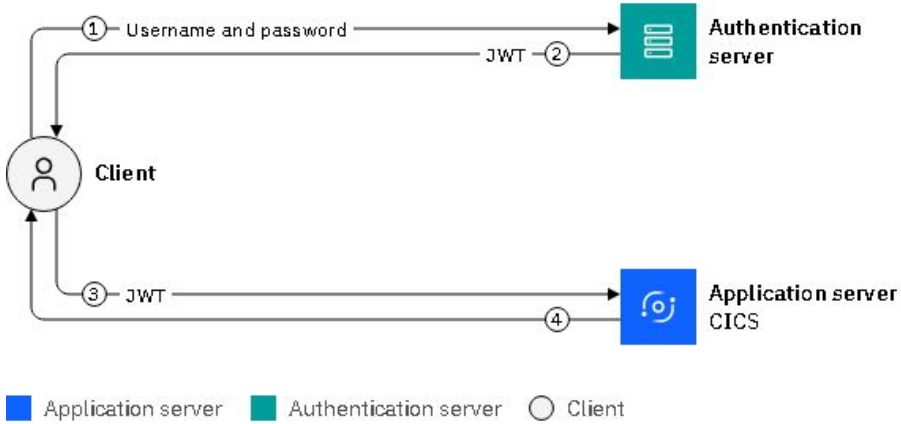
UB Social

JWT

Implementar JSON Web Token

Conceito

- JWT (JSON Web Token) é padrão (RFC 7519) de token stateless (geralmente do tipo Bearer token);
- Usado para autenticação/autorização na troca de informações entre cliente e servidor (HTTP request/response) de forma segura (geralmente em conjunto com protocolo OAuth2), assinadas com Message Authentication Code (HMAC);
- **Access Token** é token de curta duração com expiração, usado para acessar recursos protegidos da API, enviado em cada request no header Authorization;
- **Refresh Token** é token de longa duração de expiração, usado apenas para gerar novos access tokens.



Implantação

- Estrutura de token JWT consiste em:
  - **Header** (tipo de token e algoritmo de assinatura);
  - **Payload** (dados claims, como subject, issuedAt, expiration);
  - **Signature** (assinatura criptográfica do header e payload usando secret key).
- JWT amplamente utilizado em sistemas web RESTful (arquitetura REST) stateless (sem manter estado no servidor), com funções encoder (criação) e decoder (leitura).

Encoded


```
eyJhbGciOiJSUzI1NiJ9.eyJpc3MiOiJpZGciLCJzdWIiO
iJjbj1KZWFuTGVjbGVyYyxvdT1lbXBsb3llZXMsbz1pYm0
sYz1mciIsImF1ZCI6InVybjpteUVudGl0eSIsImV4cCI6M
TU0NDE5NjA0NSwiaWF0IjoxNTQ0MTg4ODQ1fQ.a6DcawLi
6p87vW1Jr1VN10oAE6gAYlOZSRtL7Z1wCoavZsJCL4ZHjW
DxTfjJu0WG0aP3Q4_cgpHjLw7GCbc55lkQSMe64nJRxl--
7ZIyEkhMpPHA_QHK7Udr9JS-SBJ4e0BCbJwk46d7D-qADd
XSzSDXmpl25GwW6HDs4JqIguRDTNpH3Z3R_5HWhitpz2rY
Vnl2XQ1VbihuqoeBBtEHKLjma0U0J3sY5Hq2stjLrW5HuL
uTfBbuJsWvlSJMqAieOAi6d1pEUH2cZ-U6AhHjFi3c_eKV
5kwCX8llvEIZC8AGwtOE2VDNmuz_NDOTy9JUYDmL63EgzJ
NaFPV1y29WdA
```


Decoded

Header

```
{
  "alg" : "RS256"
}
```

Payload

```
{
  "iss" : "idg",
  "sub" : "cn=JeanLeclerc,ou=employees,o=ibm,c=fr",
  "aud" : "urn:myEntity",
  "exp" : "1544196045",
  "iat" : "1544188845",
}
```

Verify Signature

```
RSASHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  Enter Public key or Certificate,
  Enter Private key
)
```

Fluxo de projeto (Spring Boot Kotlin)

- Cliente registra suas credenciais (/api/auth/register), acessa (/api/auth/login), recebe tokens, depois envia access token na request protegida (/api/hello). Filtro valida token, liberando acesso ao controller. Nesse exemplo simples, refresh token é gerado apenas para demonstração (ideal é gerar em /api/auth/refresh).

1. **SecurityConfig.kt** define regras de segurança, libera /api/auth/** (register e login) e protege demais endpoints, além de registrar o filtro JWT;
2. **JwtAuthenticationFilter.kt** intercepta todas requisições, lê header Authorization, valida token e define usuário no SecurityContext;
3. **JwtService.kt** gera access e refresh token, e extrai username do token;
4. **RegisterRequest.kt** define formato da requisição de cadastro com username e password;
5. **LoginRequest.kt** define formato da requisição de login com username e password;
6. **TokenResponse.kt** define formato da resposta contendo accessToken e refreshToken;
7. **UserService.kt** gerencia usuários em memória, permitindo cadastro (register) e validação de credenciais no login;
8. **AuthController.kt** expõe /api/auth/register para cadastrar usuário e /api/auth/login para autenticar, validar credenciais via UserService, gerar tokens com JwtService e retornar a resposta;
9. **HelloController.kt** expõe endpoint /api/hello que está protegido pelo Spring Security e exige token válido;
10. **HelloResponse.kt** define formato de resposta do endpoint protegido ou público de teste.

Implantação (pré-requisitos)

- JDK 21 (compilador Java);
- Kotlin (compilador Kotlin);
- Gradle (gerenciador de dependências);
- IntelliJ IDEA (editor de código);
- Postman (cliente consumidor de API).

Criação do projeto

- Projeto Spring Boot Kotlin API REST para implementação de JWT. Nesse exemplo simples, refresh token é gerado apenas para demonstração (ideal é gerar em /api/auth/refresh).
- Criar projeto em https://start.spring.io
  - Project: Gradle - Kotlin
  - Language: Kotlin
  - Spring Boot: 4.0.6
  - Project Metadata:
    - Group: com.example
    - Artifact: myjwt
    - Package name: com.example.myjwt
    - Packaging: Jar
    - Configuration: properties
    - Java: 21
  - Generate
- Extrair e abrir projeto no editor (IntelliJ IDEA)

Configurações inciais

- Criar packages do projeto:
  - com.example.myjwt.controller
  - com.example.myjwt.dto
  - com.example.myjwt.security
  - com.example.myjwt.service
- Atualizar todo conteúdo de configurações do projeto, em 'build.gradle.kts

API REST sem autenticação

1. No pacote 'myjwt.dto', criar DTO 'HelloResponse.kt';
2. No pacote 'myjwt.controller', criar Controller 'HelloController.kt';
3. Em 'build.gradle.kts', comentar: '//implementation("org.springframework.boot:spring-boot-starter-security")';
4. Construir projeto: ./gradlew clean build;
5. Executar projeto: ./gradlew bootRun
6. Acessar no browser (GET): http://localhost:8080/api/hello
    a. Response com sucesso (200 OK)

Implantação de JWT em API REST

1. Em 'build.gradle.kts', remover comentário: 'implementation("org.springframework.boot:spring-boot-starter-security")'
2. No pacote 'myjwt.security', criar Service 'JwtService.kt';
3. No pacote 'myjwt.security', criar classe 'SecurityConfig.kt';
4. No pacote 'myjwt.security', criar classe de filtro 'JwtAuthenticationFilter.kt';
5. No pacote 'myjwt.dto', criar DTO 'RegisterRequest.kt';
6. No pacote 'myjwt.dto', criar DTO 'LoginRequest.kt';
7. No pacote 'myjwt.dto', criar DTO 'TokenResponse.kt';
8. No pacote 'myjwt.service', criar repositório Service 'UserService.kt';
9. No pacote 'myjwt.controller', criar Controller 'AuthController.kt';
10. Construir projeto: ./gradlew clean build
11. Executar projeto: ./gradlew bootRun

API REST requests com JWT

1. Testar request sem autenticação: GET http://localhost:8080/api/hello
    - Response com erro (401 Unauthorized)
2. Registrar novo usuário: POST http://localhost:8080/api/auth/register

```
{
  "username": "admin",
  "password": "123"
}
```

3. Acessar e criar token JWT (copiar token gerado): POST http://localhost:8080/api/auth/login

```
{
  "username": "admin",
  "password": "123"
}
```

4. Testar request com autenticação: GET http://localhost:8080/api/hello (Authorization: Bearer token)
    - Response com sucesso (200 OK)

UB Social

ubsocial.github.io